```
import java.util.*;
class solution {
    public int solution(String[] crypt) {
        String word1 = crypt[0];
        String word2 = crypt[1];
        String word3 = crypt[2];
        Set<Character> uniq = new HashSet<>();
*1      if(uniq.size() > 10) return 0;
*2      List<Character> letter = new
                    ArrayList<>(uniq);
*3      int[] ~~lettertodig~~ arr = new int [26]
        Arrays.fill(arr, -1);
        boolean[] us = new boolean [10];
        Set<Character> nz = new HashSet<>();
        if(word1.length() > 1) nz.add(word1.charAt
                                (0));
        if(word2.length() > 1) nz.add(word2.charAt(0));
        if(word3.length() > 1) nz.add(word3.charAt(0));
        return dfs(0, letter, arr, us,
                    nz, word1, word2, word3);
    }
```

```
private int dfs (int ind, List<character> letters,
                 boolean[] us,
                 Set<character> nz,
                 String w1, String w2, String w3,
                 int[] arr) {
    if (index == letters.size()) {
        return check(w1, w2, w3, arr) ? 1 : 0
    }

    int count = 0;
    char let = letters.get(ind);

    for (int digit = 0; digit <= 9; digit++) {
        if (us[dig]) continue;
        if (digit == 0 && nz.contains(let)) continue;
        us[digit] = true;
        arr[let - 'A'] = -1;
    }
    return count;
}
private boolean check(String w1, String w2, String w3,
                      int[] map) {
    int carry = 0;
    int i = w1.length() - 1;
    int j = w2.length() - 1;
    int k = w3.length() - 1;
```

```
while (i >=0 || j >=0 || k >=0) {
    int d1 = (i>=0) ? map[w1.charAt(i) - 'A']: 0;
    int d2 = (j>=0) ? map[w2.charAt(i) - 'A']: 0;
    int d3 = (k>=0) ? map[w3.charAt(i) - 'A']: 0;
    int sum = d1 + d2 + carry;
    if (sum % 10 != d3) return false;
    carry = sum / 10
    i--;
    j--;
    k--;
  }
  return carry == 0;
 }
}
```

Step 1 extract words from array

w1 = "SEND"
w2 = "MORE"
w3 = "MONEY"

so SEND + MORE = MONEY

Step 2

collect all unique letters.

S, E, N, D, M, O, R, Y = total = 8

// stored in uniq.

*1 => if uniq.size > 10 return false;

*2 => store them in list letters

letters = {S, E, N, D, M, O, R, Y}

Step 3

*3 => arr[26] as we for storing what numbers we choose for each letter

so we will fill with -1 initially

now I want to know what digits are used so I use a arr[10] for digits 0-9 I need it to be either true or false so boolean array of size 10

Step 4

first letter can't be zero

accessing them and storing in set

or we can do it with array of size 3

later optimization

now set = {S, M}

Step 5 backtracking

dfs(0) // assign a digit to letter(0)

}

~~check~~

1) check $S = (\overset{\times}{0}, \overset{\checkmark}{1}, 2, 3, 4, 5, 6, 7, 8, 9)$

S cannot be 0

try 1

$S = 1$ // update in ans $[\overset{\text{th}}{19} - 1] = 1$

18

vis[1] = true

Call dfs(1) $S = (0, \overset{\times}{1}, 2, 3, 4, 5, 6, 7, 8, 9)$

e = 0

dfs(2)

				$\overset{\times}{0}$			
S	E	N	D	M	O	R	Y
1	0	2	3	4	5	6	7

recursion continues

N =

simulate addition

```
  SEND
  MORE
 -----
 MONEY
```

D + E = Y

3 + 0 = 7

false

Y = 8

still invalid.

invalid